

A NOVEL METHODOLOGY OF DESIGN AND DEVELOPMENT OF DEVTINDER: A DEVELOPER NETWORKING & MATCHMAKING PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

NAVIN KUMAR
CHANDI CHARAN MAHTO

22050440067
22050440034

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

2026

DECLARATION CERTIFICATE

We hereby declare that the project entitled "**DevTinder - A Developer Networking and Matchmaking Platform**" is an authentic record of our own work carried out as requirements of the Final Year Project for the award of Bachelor of Engineering in Computer Science and Engineering from Cambridge Institute of Technology, Ranchi (Affiliated to Jharkhand University of Technology), during the academic year 2025-26, under the guidance of **Prof. Deepak Kumar Verma**, Department of Computer Science and Engineering.

To the best of our knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA

(Head of Department)

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "**DevTinder - A Developer Networking and Matchmaking Platform**" is a bonafide record of work carried out by **Navin Kumar (Roll No. 22050440067)** and **Chandi Charan Mahto (Roll No. 22050440034)**, students of 8th Semester, Bachelor of Engineering in Computer Science and Engineering, Cambridge Institute of Technology, Ranchi, during the academic year 2025-26.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

We take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. We would like to express my sincere thanks to **Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering)** and to my guide **Prof. Deepak Kumar Verma**, whose help in the course of development of this manuscript is invaluable. As our supervisor, he has constantly encouraged me to remain focused on achieving our goal.

We would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. We have been fortunate to have a wonderful support structure among fellow students. Our sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. We are truly indebted. Last but not least, We would like to dedicate this work to Our family for their love and understanding.

NAVIN KUMAR

22050440067

CHANDI CHARAN MAHTO

22050440034

ABSTRACT

In today's rapidly evolving software industry, developer collaboration and professional networking are critical to career advancement. While general-purpose networking platforms exist, none are purpose-built for software developers seeking peer discovery based on technical skills and shared interests. DevTinder addresses this gap by providing a dedicated, skill-first networking platform exclusively designed for the developer community.

DevTinder is a full-stack web application that enables developers to create detailed technical profiles, browse a dynamic discovery feed of other developer profiles, express interest or disinterest, establish mutual connections, and engage in real-time one-on-one text conversations. The platform adopts a matchmaking paradigm - a connection is only established when both users express interest in each other, ensuring quality interactions between willing participants.

The frontend is a Single Page Application (SPA) built with React.js (v19), using Redux Toolkit for global state management and React Router v7 for client-side navigation. Styling is handled with Tailwind CSS and DaisyUI.

The backend is a RESTful API server built with Node.js and Express.js, sharing its HTTP server with a Socket.io WebSocket server for real-time communication. Authentication uses JSON Web Tokens (JWT) stored as HTTP-only cookies, with bcrypt for password hashing.

This report presents the complete design, implementation, and testing of the DevTinder platform, developed as a Final Year Project at Cambridge Institute of Technology, Ranchi.

Keywords: *Full Stack Web Application, React.js, Node.js, Express.js, MongoDB, Socket.io, Redux Toolkit, JWT, REST API, Real-time Chat, Developer Networking.*

TABLE OF CONTENTS

Declaration	
Certificate	
Acknowledgement	
Abstract	
List of Tables.....	3
List of Figures	4
Chapter 1: Introduction.....	5
1.1 Background and Motivation	5
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Scope of the Project	7
1.6 Report Organisation	7
Chapter 2: Literature Review	8
2.1 Review of Existing Platforms.....	8
2.2 Comparison of Existing Systems	9
2.3 Limitations of Existing Systems	10
2.4 Proposed System.....	10
Chapter 3: System Requirements	11
3.1 Functional Requirements.....	11
3.2 Non-Functional Requirements.....	12
3.3 Hardware Requirements	12
3.4 Software Requirements	13
Chapter 4: System Architecture	14
4.1 Overall Architecture.....	14
4.2 Client-Server Communication	15
4.3 Technology Stack Overview	16
Chapter 5: Frontend Design and Implementation	17
5.1 Technology Stack	17
5.2 Application Routing.....	18
5.3 Redux State Management.....	18
Chapter 6: Backend Design and Implementation.....	19
6.1 Technology Stack	19

6.2 Middleware Architecture.....	20
6.3 Route Modules.....	21
6.4 Business Logic.....	21
Chapter 7: Database Design.....	22
7.1 Database Selection Rationale	22
7.2 Entity Overview.....	22
7.3 User Model	23
7.4 ConnectionRequest Model	23
Chapter 8: Real-Time Communication.....	24
8.1 Socket.io Architecture.....	24
8.2 Chat Room Design.....	24
8.3 Event Flow.....	25
Chapter 9: Authentication and Security.....	26
9.1 Authentication Flow.....	26
9.2 Password Security.....	27
9.3 Input Validation	27
9.4 CORS and Environment Security	28
Chapter 10: API Reference	29
10.1 Authentication Routes.....	29
10.2 Profile Routes	29
10.3 Connection Request Routes.....	29
Chapter 11: Testing	30
11.1 Testing Strategy	30
11.2 Unit Testing	30
11.3 Integration Testing	31
Chapter 12: Project Folder Structure	35
12.1 Backend Structure.....	35
12.2 Frontend Structure	39
Chapter 13: Conclusion and Future Scope.....	41
13.1 Conclusion.....	41
13.2 Future Scope	42
References	44

LIST OF TABLES

Table 2.1	Comparison of Existing Networking Platforms.....	9
Table 3.1	Functional Requirements.....	11
Table 3.2	Non-Functional Requirements.....	12
Table 3.3	Hardware Requirements	12
Table 3.4	Software Requirements	13
Table 4.1	System Architecture Layers.....	14
Table 4.2	Complete Technology Stack.....	16
Table 5.1	Frontend Technology Stack.....	17
Table 5.2	Application Routes.....	18
Table 5.3	Redux State Slices.....	18
Table 6.1	Backend Technology Stack	19
Table 6.2	Backend Route Modules.....	21
Table 7.1	User Model Schema	23
Table 7.2	ConnectionRequest Model Schema	23
Table 8.1	Socket.io Events Reference	25
Table 10.1	Authentication API Endpoints	29
Table 10.2	Profile API Endpoints	29
Table 10.3	Connection Request Endpoints	29
Table 11.1	Unit Test Cases	30
Table 11.2	Integration Test Cases	31

LIST OF FIGURES

Figure 1	Home Page of DevTinder	32
Figure 2	Create Account Page.....	32
Figure 3	My Profile Page	33
Figure 4	Search to Find Developers	33
Figure 5	List of Developers	34

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The global software development industry has grown enormously over the past two decades. With millions of developers distributed across geographic boundaries, discovering and connecting with peers who share complementary skills and professional interests has become increasingly important. Whether for collaboration on side projects, open-source contributions, hackathons, or simply building a professional network, developer-to-developer connections drive both individual career growth and industry innovation.

Existing general-purpose professional networking platforms, while widely used, are not designed with the specific needs of software developers in mind. They prioritise job searching and corporate branding over technical skill discovery. Developer-centric platforms like GitHub facilitate code collaboration but lack structured networking and messaging features. This creates a clear gap for a platform that places technical skills at the centre of the discovery and connection experience.

DevTinder was conceived and developed at Cambridge Institute of Technology, Ranchi, as a Final Year Project to bridge this gap. By adopting an intuitive, feed-based profile discovery mechanism combined with a mutual interest matching model and integrated real-time chat, DevTinder provides a compelling, purpose-built networking experience for the developer community.

1.2 Problem Statement

The following key problems motivate the development of DevTinder:

- Existing professional networks are too broad and do not surface developers based on specific technical skills, making it difficult to find relevant collaborators.
- There is no unified platform that integrates developer profile discovery, a mutual interest connection model, and real-time messaging in a single cohesive application.
- Generic networking platforms expose users to unsolicited contact. A mutual interest model ensures both parties are willing to communicate before a connection is established.
- Developers lack a dedicated space to present their technical identity - skills, technologies, and professional bio - as the primary focus of their profile.

1.3 Objectives

1. To design and implement a full-stack web application for developer networking using modern, industry-standard technologies.
2. To provide a dynamic, paginated feed where developers can browse profiles and express interest or disinterest.
3. To implement a bidirectional connection request system requiring mutual acceptance before messaging is enabled.
4. To enable real-time one-on-one chat between connected developers using WebSockets via Socket.io.
5. To build a secure, stateless authentication system using JWT and bcrypt password hashing.
6. To deliver a responsive and accessible user interface using React.js, Tailwind CSS, and DaisyUI.
7. To implement automated background task scheduling via node-cron and email delivery via AWS SES.

1.4 Scope of the Project

DevTinder is scoped as a browser-based Single Page Application on all modern browsers. The frontend is a React.js SPA served via a Vite dev server. The backend is a Node.js/Express.js REST API server that also hosts a Socket.io WebSocket server on the same port. MongoDB is used as the primary database, deployable locally or on MongoDB Atlas.

The system supports the complete developer networking lifecycle: registration, profile creation, feed-based discovery, connection request management, real-time chat, and account management. Email notifications for key events are supported via AWS SES. Scheduled background jobs are implemented using node-cron. Cloud deployment, native mobile apps, and advanced analytics are identified as future scope and are not included in the current deliverable.

1.6 Report Organisation

This report is organised into thirteen chapters. Chapter 2 presents a literature review of existing platforms. Chapter 3 documents system requirements. Chapter 4 describes the overall architecture. Chapters 5 and 6 detail the frontend and backend implementation. Chapter 7 covers database design. Chapter 8 explains real-time communication. Chapter 9 addresses authentication and security. Chapter 10 provides a complete API reference. Chapter 11 documents testing. Chapter 12 presents the folder structure. Chapter 13 concludes with future scope.

CHAPTER 2: LITERATURE REVIEW

2.1 Review of Existing Platforms

2.1.1 LinkedIn

LinkedIn is the world's largest professional networking platform with over 950 million members as of 2024. It allows users to create professional profiles, connect with colleagues, endorse skills, and apply for jobs. While comprehensive, LinkedIn is designed for broad professional networking and does not provide a skill-based discovery feed or a mutual interest matching model. The connection model is one-directional and the feed is driven by engagement metrics rather than technical compatibility.

2.1.2 GitHub

GitHub is the world's leading platform for version control and open-source collaboration with over 100 million developers. While developers can follow each other and star repositories, GitHub is fundamentally a code hosting platform. It does not provide a structured connection system, a discovery feed based on skills, or a real-time direct messaging feature between individuals.

2.1.3 Stack Overflow

Stack Overflow is the most widely used question-and-answer platform for programmers. Users build reputation through answered questions and upvoted content. While it builds community and identifies subject matter experts, it is not a networking platform in the traditional sense and does not support direct messaging or profile-based peer discovery.

2.1.4 Discord

Discord, originally for gaming, has become a popular platform for developer communities. Many open-source projects maintain Discord servers for group discussion and real-time chat. However, Discord operates on a community-first server model rather than a person-first discovery model. It does not offer a profile-based developer discovery feed or a structured one-to-one connection lifecycle.

2.1.5 Hashnode and Dev.to

Hashnode and Dev.to are developer-focused blogging platforms. They allow developers to write technical articles, follow other writers, and build personal brand through content. While they provide a developer-specific context, they are not networking platforms and do not support direct messaging or structured connection management.

2.2 Comparison of Existing Systems

Feature	LinkedIn	GitHub	Stack Overflow	Discord	DevTinder
Skill-based discovery feed	No	No	No	No	Yes
Mutual interest model	No	No	No	No	Yes
Developer-specific profiles	Partial	Yes	Partial	No	Yes
Real-time direct messaging	Yes	No	No	Yes	Yes
Structured connection lifecycle	Yes	No	No	No	Yes
Integrated chat + discovery	No	No	No	No	Yes

Table 2.1: Comparison of Existing Networking Platforms

2.3 Limitations of Existing Systems

- No existing platform combines skill-based developer discovery with an interactive feed and integrated real-time messaging.
- General networking platforms do not prioritise technical skills as the primary discovery criterion.
- Platforms with messaging do not enforce a mutual interest requirement, leading to unsolicited contact.
- No single platform provides an end-to-end flow from profile discovery through mutual connection to persistent real-time chat in one unified application.

2.4 Proposed System

DevTinder is proposed as a purpose-built platform that integrates developer profile discovery, bidirectional connection lifecycle management, and real-time messaging into a single cohesive web application. The mutual interest model differentiates DevTinder from all existing platforms. The skill-first profile design ensures technical compatibility is central to discovery. The full-stack implementation using React.js, Node.js, MongoDB, and Socket.io provides a modern, scalable, and maintainable technical foundation suitable for real-world deployment.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Functional Requirements

The following functional requirements define what the DevTinder system shall do. Each requirement is assigned a unique identifier and a priority level (High, Medium, or Low).

FR-01 (High): The system shall allow new users to register using first name, last name, email, and password.

FR-02 (High): The system shall authenticate registered users using email and password credentials.

FR-03 (High): The system shall issue a JWT stored as an HTTP-only cookie on successful authentication.

FR-04 (High): Users shall be able to view their own profile details after logging in.

FR-05 (High): Users shall be able to edit their profile including skills, bio, photo URL, age, and gender.

FR-06 (High): The system shall display a paginated feed of developer profiles not yet interacted with.

FR-07 (High): Users shall be able to send an 'interested' or 'ignored' request to any profile in the feed.

FR-08 (High): Users shall be able to view all pending connection requests they have received.

FR-09 (High): Users shall be able to accept or reject a received connection request.

FR-10 (Medium): Users shall be able to view their complete list of accepted connections.

FR-11 (High): Connected users shall be able to exchange real-time text messages.

FR-12 (High): All chat messages shall be persisted in the database for retrieval in future sessions.

FR-13 (High): The system shall allow users to log out and invalidate their session cookie.

FR-14 (Medium): The system shall send email notifications for key events via AWS SES.

FR-15 (Low): The system shall execute scheduled background maintenance jobs via node-cron.

3.2 Non-Functional Requirements

The following non-functional requirements specify the quality attributes and constraints the system must satisfy.

NFR-01 [Performance]: The initial application page shall load within 3 seconds on a standard broadband connection.

NFR-02 [Performance]: Chat messages shall be delivered to recipients within 500 milliseconds under normal conditions.

NFR-03 [Security]: All user passwords shall be hashed with bcrypt (salt rounds ≥ 10) before storage.

NFR-04 [Security]: JWT tokens shall be stored as HTTP-only cookies to prevent cross-site scripting access.

NFR-05 [Security]: All user inputs shall be validated on both client and server sides.

NFR-06 [Usability]: The application shall be fully responsive for screens from 360px to 1920px width.

NFR-07 [Reliability]: All API endpoints shall return appropriate HTTP status codes and descriptive error messages.

NFR-08 [Maintainability]: The codebase shall follow a modular, maintainable folder and file structure.

NFR-09 [Security]: All sensitive credentials and configuration shall be stored in environment variables only.

NFR-10 [Scalability]: The system shall handle concurrent WebSocket connections from multiple users reliably.

3.3 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3 (2.0 GHz)	Intel Core i5 / AMD Ryzen 5 (2.5 GHz+)
RAM	4 GB	8 GB or higher
Storage	20 GB free space	50 GB SSD
Network	Broadband (10 Mbps)	Broadband (25 Mbps or higher)
Display	1024 x 768 resolution	1920 x 1080 Full HD

Table 3.3: Hardware Requirements

3.4 Software Requirements

Software	Version	Purpose
Node.js	v20.x LTS or higher	Server-side JavaScript runtime environment
npm	v10.x or higher	Node.js package manager
MongoDB	v7.x (local or Atlas cloud)	NoSQL document database server
Git	v2.x	Distributed version control system
Visual Studio Code	Latest stable	Recommended code editor
Google Chrome / Firefox	Latest stable	Primary testing browser
Postman	Latest stable	REST API testing and documentation tool
MongoDB Compass	Latest stable	GUI client for MongoDB inspection

Table 3.4: Software Requirements

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 Overall Architecture

DevTinder is built on a three-tier client-server architecture comprising a Presentation Tier (React.js SPA), an Application Tier (Node.js/Express.js API server), and a Data Tier (MongoDB). An additional real-time communication layer is provided by Socket.io, which operates as a WebSocket upgrade layer on the same HTTP server as the REST API, eliminating the need for a separate port or server process.

The frontend and backend are maintained as completely separate codebases communicating exclusively through well-defined REST API contracts and Socket.io event contracts. This decoupled architecture enables independent development and testing of each tier and facilitates future horizontal scaling.

Layer	Technology	Responsibility
Presentation	React.js 19 + Tailwind CSS + DaisyUI	UI rendering, user interactions, client-side routing
State Management	Redux Toolkit + React-Redux	Centralised global client-side application state
API Client	Axios	HTTP request construction and response handling
Application (REST)	Node.js + Express.js 4.x	Business logic, REST API routing and middleware
Real-time	Socket.io 4.x	WebSocket server for bidirectional chat events
Data	MongoDB + Mongoose 8.x	Persistent storage, schema enforcement and ODM
Authentication	JSON Web Token + bcrypt	Stateless token-based auth and password hashing

Table 4.1: System Architecture Layers

4.2 Client-Server Communication

4.2.1 HTTP REST API

All CRUD operations, authentication flows, profile management, connection lifecycle management, and data retrieval are performed via standard HTTP REST API calls using Axios on the frontend. The backend API server listens on port 7777. All requests to protected resources automatically include the JWT cookie. The API returns JSON responses with appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error).

4.2.2 WebSocket via Socket.io

Real-time chat uses persistent WebSocket connections managed by Socket.io. The Socket.io server is initialised on the same HTTP server instance as Express.js, sharing port 7777. When a user opens a chat window, the frontend establishes a Socket.io connection, joins a deterministic chat room identified by a sorted combination of both user IDs, and listens for incoming messageReceived events. Messages are persisted to MongoDB on every send.

4.3 Technology Stack Overview

Category	Technology	Version
Frontend Framework	React.js	19.0.0
Build Tool	Vite	6.1.0
CSS Framework	Tailwind CSS + DaisyUI	3.4.17 + 4.12.23
State Management	Redux Toolkit	2.5.1
HTTP Client	Axios	1.7.9
Client Routing	React Router DOM	7.1.5
Backend Runtime	Node.js	20.x LTS
Backend Framework	Express.js	4.19.2
Database	MongoDB + Mongoose	7.x + 8.6.1
Real-time	Socket.io	4.8.1
Authentication	jsonwebtoken + bcrypt	9.0.2 + 5.1.1
Email Service	AWS SES SDK v3	3.716.0
Scheduling	node-cron	3.0.3
Validation	validator.js	13.12.0
Environment Config	dotenv	16.4.7

Table 4.2: Complete Technology Stack

CHAPTER 5: FRONTEND DESIGN AND IMPLEMENTATION

The frontend of DevTinder is a Single Page Application built with React 19. Vite serves as the build tool and development server, providing near-instantaneous hot module replacement (HMR). Styling is managed through Tailwind CSS utility classes augmented by DaisyUI component classes. Global application state is managed with Redux Toolkit. All HTTP communication with the backend is handled through Axios.

5.1 Technology Stack

Package	Version	Purpose
react	^19.0.0	Core UI component library
react-dom	^19.0.0	React DOM rendering engine
react-router-dom	^7.1.5	Declarative client-side routing
@reduxjs/toolkit	^2.5.1	Simplified predictable state management
react-redux	^9.2.0	React bindings for the Redux store
axios	^1.7.9	Promise-based HTTP client for API requests
tailwindcss	^3.4.17	Utility-first CSS framework
daisyui	^4.12.23	Tailwind CSS component library
vite	^6.1.0	Frontend build tool and development server

Table 5.1: Frontend Technology Stack

5.2 Application Routing

React Router v7 manages all client-side navigation. Routes are defined in App.jsx and nested under the Body component, which acts as an authentication guard: on mount it calls GET /profile/view to check authentication and dispatches user data to the Redux store. If the API call fails (401 Unauthorized), the user is redirected to /login.

React Router v7 manages all client-side navigation. The application defines the following routes, each mapped to a dedicated React component with appropriate access control.

Route "/" renders the Feed component (Protected): Developer discovery feed with paginated UserCards.

Route "/login" renders the Login component (Public): Combined login and sign-up form page.

Route "/profile" renders the Profile component (Protected): View and navigate to edit own developer profile.

Route "/connections" renders the Connections component (Protected): Grid view of all accepted connections.

Route "/requests" renders the Requests component (Protected): List of all pending received connection requests.

5.3 Redux State Management

Redux Toolkit manages all global application state through four independent slices. The centralised store eliminates prop drilling across deeply nested components and ensures consistent state across all views. Each slice is defined in its own file in src/utills/ using the createSlice API.

Redux Toolkit manages all global application state through four independent slices, each with a clearly defined purpose.

The userSlice holds Logged-in user's complete profile. Key actions are addUser, removeUser. It is consumed by: Body, Navbar, Profile, EditProfile.

The feedSlice holds Array of developer profiles in feed. Key actions are addFeed, removeUserFromFeed. It is consumed by: Feed, UserCard.

The connectionSlice holds Array of accepted connection objects. Key actions are addConnections. It is consumed by: Connections.

The requestSlice holds Array of pending received requests. Key actions are addRequests, removeRequest. It is consumed by: Requests.

CHAPTER 6: BACKEND DESIGN AND IMPLEMENTATION

The backend is a Node.js application built on Express.js. It serves as both a RESTful HTTP API server and a Socket.io WebSocket server, both running on the same HTTP server instance on port 7777. The application follows a modular, domain-driven architecture with clear separation between models, routes, middleware, and utilities. All configuration is managed through environment variables loaded by dotenv.

6.1 Technology Stack

Package	Version	Purpose
express	^4.19.2	HTTP web framework for REST API routing and middleware
mongoose	^8.6.1	MongoDB Object Document Mapper with schema support
jsonwebtoken	^9.0.2	JWT token generation and cryptographic verification
bcrypt	^5.1.1	Secure adaptive password hashing with automatic salting
socket.io	^4.8.1	Real-time bidirectional WebSocket event system
cors	^2.8.5	Cross-Origin Resource Sharing configuration
@aws-sdk/client-ses	^3.716.0	AWS Simple Email Service client (v3 SDK)
node-cron	^3.0.3	Cron expression-based background task scheduler

Table 6.1: Backend Technology Stack

6.2 Middleware Architecture

6.2.1 CORS Middleware

Configured to allow cross-origin requests exclusively from `http://localhost:5173` (the Vite frontend server) with `credentials: true`. This allows the browser to send the authentication cookie with every cross-origin API request. Requests from all other origins are rejected.

6.2.2 `express.json()`

Parses incoming request bodies with Content-Type `application/json` and populates `req.body`. Required for all POST and PATCH requests that send data in the request body.

6.2.3 `cookie-parser`

Parses the HTTP Cookie header and makes all cookie values available on `req.cookies`. The authentication middleware relies on this to extract the JWT stored in the cookie named `'token'`.

6.2.4 Custom auth Middleware

Applied at the route level on all protected routes. Reads `req.cookies.token`, verifies the JWT signature using `jwt.verify()` with `JWT_SECRET`, decodes the payload to extract the user's MongoDB `_id`, queries the User collection for the complete user document, and attaches it to `req.user`. If the token is absent, malformed, or expired, returns HTTP 401 Unauthorized immediately.

6.3 Route Modules

Backend routes are organised into independent Express Router modules mounted in app.js. Each module handles a specific domain of the application.

Module	File Path	Key Endpoints
Auth Router	src/routes/auth.js	POST /signup, POST /login, POST /logout
Profile Router	src/routes/profile.js	GET /profile/view, PATCH /profile/edit, PATCH /profile/password
Request Router	src/routes/request.js	POST /request/send/:status/:userId, POST /request/review/:status/:requestId
User Router	src/routes/user.js	GET /user/requests/received, GET /user/connections, GET /user/feed
Chat Router	src/routes/chat.js	GET /chat/:targetUserId

Table 6.2: Backend Route Modules

6.4 Business Logic

6.4.1 Feed Filtering Algorithm

The /user/feed endpoint queries the ConnectionRequest collection to find all user IDs that the logged-in user has already interacted with in either direction. It then queries the User collection using a \$nin (not in) operator to exclude the logged-in user and all interacted user IDs. Results are paginated using skip and limit to support infinite scroll on the frontend.

6.4.2 Connection Request Validation

Before persisting a new ConnectionRequest, the backend validates: that the target user exists; that the status is either 'interested' or 'ignored'; that fromUserId is not equal to toUserId; and that no existing request exists between the two users in either direction. Violations return appropriate 4xx HTTP error responses.

CHAPTER 7: DATABASE DESIGN

7.1 Database Selection Rationale

MongoDB was selected as the primary database for DevTinder for the following reasons:

- **Document Model Flexibility:** MongoDB's document model allows optional fields (skills, age, gender) to be absent without NULL columns or schema migrations, which is beneficial during iterative development.
- **Mongoose ODM:** Mongoose adds schema definition, field-level validation, instance methods, and middleware hooks on top of MongoDB's native driver.
- **Native JSON Compatibility:** Data flows from MongoDB through Node.js to React.js entirely as JSON-compatible objects, eliminating serialisation transformations.
- **Scalability:** MongoDB's horizontal sharding capability makes it suitable for growing user and message data volumes.
- **Community and Tooling:** MongoDB Atlas provides a free cloud-hosted tier ideal for academic projects, and MongoDB Compass provides a GUI for schema inspection.

7.2 Entity Overview

DevTinder's database contains three primary collections: User, ConnectionRequest, and Chat. The User collection is the central entity. All other collections reference User documents by their MongoDB ObjectId. ConnectionRequest models the directed stateful relationship between two users. Chat stores the complete conversation history between any connected pair as embedded message objects.

7.3 User Model

The User model is the core entity of the DevTinder system. Every other collection references it. The schema includes built-in validators powered by validator.js and uses Mongoose timestamps for automatic createdAt and updatedAt fields.

The User model defines the following fields in its Mongoose schema. Each field is described with its data type, constraints, and purpose.

- "firstName" (String): Developer's first name. Constraints: Required, min: 4, max: 50.
- "lastName" (String): Developer's last name. Constraints: Optional.
- "emailId" (String): Unique login email address. Constraints: Required, unique, lowercase, isEmail().
- "password" (String): bcrypt-hashed password, never plain text. Constraints: Required, isStrongPassword().

7.4 ConnectionRequest Model

The ConnectionRequest model captures the directed relationship between two users and tracks the current state. A compound unique index on { fromUserId, toUserId } prevents duplicate requests. A pre-save hook validates that the two IDs are not identical.

The ConnectionRequest model captures the directed relationship between two users with the following fields.

- "fromUserId" (ObjectId): ID of the user who initiated the request. Constraints: Required, ref: 'User'.
- "toUserId" (ObjectId): ID of the user who received the request. Constraints: Required, ref: 'User'.
- "status" (String): Current status of the connection lifecycle. Constraints: Required, Enum: ignored / interested / accepted / rejected.
- "createdAt" (Date): Timestamp when request was created. Constraints: Auto (timestamps).
- "updatedAt" (Date): Timestamp of last status change. Constraints: Auto (timestamps).

CHAPTER 8: REAL-TIME COMMUNICATION

8.1 Socket.io Architecture

Real-time chat in DevTinder is implemented using Socket.io, which provides reliable event-based bidirectional communication built on the WebSocket protocol with automatic HTTP long-polling fallback for restricted environments. The Socket.io server is initialised in `src/utils/socket.js` by passing the Node.js HTTP server instance to `new Server(httpServer, options)`. This allows Socket.io to intercept WebSocket upgrade requests on port 7777, sharing it with the Express REST API without requiring a separate server process.

8.2 Chat Room Design

Each conversation between two users takes place in a dedicated Socket.io room. The room identifier is generated deterministically by sorting both participants' MongoDB user IDs lexicographically and joining them with an underscore separator:

```
Room ID = [userId1, userId2].sort().join('_')
```

This formula guarantees that both users always join the same room regardless of which user initiates the connection. When a user navigates to the chat page, the frontend emits a `joinChat` event with the target user's ID. The server computes the room ID and calls `socket.join(roomId)`. All subsequent messages for this conversation are routed exclusively within this room.

8.3 Event Flow

The real-time chat system uses the following Socket.io events for bidirectional communication between client and server.

"joinChat" (Client to Server): Client requests to join the room with the specified user.

Payload: targetUserId: String.

"sendMessage" (Client to Server): Client sends a message; server persists and broadcasts. Payload: targetUserId: String, text: String.

"messageReceived" (Server to Client): Server emits the new message to all sockets in the room. Payload: senderId: String, text: String, createdAt: Date.

On receiving a sendMessage event, the server: finds or creates the Chat document for the two participants; pushes the new message (senderId, text, createdAt) to the messages array; saves the updated Chat document to MongoDB; and emits messageReceived to all sockets in the room (including the sender) using io.to(roomId).emit(). Emitting to the sender as well ensures multi-tab and multi-device scenarios are handled correctly.

CHAPTER 9: AUTHENTICATION AND SECURITY

9.1 Authentication Flow

DevTinder uses stateless JWT-based authentication. No server-side session store is required - the authentication state is carried entirely within a signed HTTP-only cookie.

9.1.1 Registration Flow

8. User submits firstName, lastName, emailId, and password via POST /signup.
9. Mongoose validates all fields: email format via isEmail(), password strength via isStrongPassword(), and firstName length (min 4, max 50 chars).
10. The password is hashed using bcrypt.hash(password, 10). The original password is never stored.
11. A new User document is saved to MongoDB.
12. getJWT() generates a signed JWT with 7-day expiry and it is set as an HTTP-only cookie named 'token'.
13. The user document (password excluded) is returned with HTTP 201.

9.1.2 Login Flow

14. User submits emailId and password via POST /login.
15. Backend finds the User by emailId. If not found, returns 400 Bad Request.
16. validatePassword() compares the input against the stored bcrypt hash.
17. On success, a new JWT is generated and set as a cookie with the user object returned.

9.1.3 Protected Request Flow

18. The browser automatically appends the 'token' cookie to every API request.
19. The auth middleware reads req.cookies.token, calls jwt.verify(token, JWT_SECRET), and extracts the _id from the decoded payload.
20. The middleware queries MongoDB for the user and attaches the document to req.user for use in route handlers.

9.1.4 Logout Flow

21. POST /logout clears the 'token' cookie by setting it with an expiry in the past.
22. The frontend dispatches removeUser() to Redux and redirects to /login.

9.2 Password Security

Passwords are hashed using bcrypt with a salt round factor of 10. bcrypt generates a unique random salt for each password before hashing, so two identical passwords produce different hash values - eliminating rainbow table attacks. The adaptive cost factor means rounds can be increased as hardware improves to maintain brute-force resistance.

9.3 Input Validation

- Mongoose Schema Validators: emailId validated with validator.isEmail(), photoUrl with validator.isURL(), password with validator.isStrongPassword() (requires uppercase, lowercase, number, and symbol), and firstName/lastName with minlength/maxlength constraints.
- Route-level Whitelist Validation: The profile edit endpoint uses a custom validateEditProfileData() utility that whitelists allowed fields (firstName, lastName, about, skills, photoUrl, age, gender). Any attempt to update restricted fields such as emailId or isPremium is rejected with 400, preventing mass-assignment vulnerabilities.

9.4 CORS and Environment Security

CORS is configured to permit requests only from the known frontend origin with credentials: true. Scripts from other origins cannot make authenticated API calls using the user's cookie.

All sensitive configuration - MongoDB URI, JWT_SECRET, AWS IAM credentials - are stored in a .env file loaded by dotenv. This file is listed in .gitignore and never committed to version control, ensuring secrets are not exposed in the source code repository.

CHAPTER 10: API REFERENCE

All API endpoints are hosted at `http://localhost:7777` during development. All request and response bodies are JSON-encoded. Protected endpoints require the 'token' HTTP-only cookie. Standard HTTP status codes are used: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.

10.1 Authentication Routes

The Authentication routes handle user registration, login, and logout.

POST `/signup` — public endpoint. with `firstName`, `lastName`, `emailId`, `password`

Returns: 201 Created + user object + JWT cookie.

POST `/login` — public endpoint. with `emailId`, `password` Returns: 200 OK + user object + JWT cookie.

POST `/logout` — requires authentication. Returns: 200 OK + cookie cleared.

10.2 Profile Routes

The Profile routes allow authenticated users to view and update their developer profile.

GET `/profile/view` — requires authentication. Returns: 200 OK: complete user object (password excluded).

PATCH `/profile/edit` — requires authentication. with `firstName`, `lastName`, `photoUrl`, `about`, `skills`, `age`, `gender` Returns: 200 OK: updated user object.

PATCH `/profile/password` — requires authentication. with `password` (new password string) Returns: 200 OK: success message.

10.3 Connection Request Routes

The Connection Request routes manage the lifecycle of developer connections.

POST `/request/send/:status/:userId` — requires authentication. with `status`: `interested|ignored`; `userId`: target user `_id` Returns: 201 Created: connection request document.

POST `/request/review/:status/:requestId` — requires authentication. with `status`: `accepted|rejected`; `requestId`: request `_id` Returns: 200 OK: updated request document.

CHAPTER 11: TESTING

11.1 Testing Strategy

The testing of DevTinder was conducted using a multi-level strategy encompassing unit testing, integration testing, and end-to-end manual testing. Unit tests verify the correctness of individual functions and middleware in isolation. Integration tests validate complete API request-response cycles including real database interactions, using Postman as the HTTP client. End-to-end tests exercise complete user workflows through the browser from registration to real-time chat.

11.2 Unit Testing

Unit tests were designed to validate individual backend functions with dependencies mocked or isolated where necessary.

The following unit tests were designed to validate individual backend functions in isolation.

UT-01 [User Model]: `getJWT()` returns a correctly signed JWT string. Expected: 3-segment JWT string. Result: Pass.

UT-02 [User Model]: `validatePassword()` returns true for correct password. Expected: Boolean true. Result: Pass.

UT-03 [User Model]: `validatePassword()` returns false for wrong password. Expected: Boolean false. Result: Pass.

UT-04 [auth middleware]: Valid JWT cookie allows request to proceed. Expected: `next()` called, `req.user` set. Result: Pass.

UT-05 [auth middleware]: Absent JWT cookie returns 401. Expected: HTTP 401 response. Result: Pass.

UT-06 [auth middleware]: Tampered JWT returns 401. Expected: HTTP 401 response. Result: Pass.

UT-07 [validation.js]: Valid profile fields pass whitelist check. Expected: No error thrown. Result: Pass.

UT-08 [validation.js]: Disallowed field (`emailId`) throws error. Expected: Error with message. Result: Pass.

11.3 Integration Testing

Integration tests were performed using Postman collections against the live Node.js server with a test MongoDB database.

Integration tests were performed using Postman against the live Node.js server with a test MongoDB database. The following test cases were executed.

- IT-01 — POST /signup: Register new user with valid data. Expected: 201 Created. Result: Pass.
- IT-02 — POST /signup: Register with already used email. Expected: 400 Bad Request. Result: Pass.
- IT-03 — POST /signup: Register with weak password. Expected: 400 Bad Request. Result: Pass.
- IT-04 — POST /login: Login with correct credentials. Expected: 200 OK + JWT cookie. Result: Pass.
- IT-05 — POST /login: Login with incorrect password. Expected: 400 Bad Request. Result: Pass.
- IT-06 — GET /profile/view: Fetch profile with valid JWT cookie. Expected: 200 OK + user object. Result: Pass.
- IT-07 — GET /profile/view: Fetch profile without any cookie. Expected: 401 Unauthorized. Result: Pass.
- IT-08 — PATCH /profile/edit: Update skills array with valid data. Expected: 200 OK + updated user. Result: Pass.
- IT-09 — PATCH /profile/edit: Attempt to update emailId (disallowed). Expected: 400 Bad Request. Result: Pass.
- IT-10 — POST /request/send/interested/:id: Send interested request to valid user. Expected: 201 Created. Result: Pass.
- IT-11 — POST /request/send/interested/:id: Send request to own user ID. Expected: 400 Bad Request. Result: Pass.
- IT-12 — POST /request/send/interested/:id: Send duplicate request to same user. Expected: 400 Bad Request. Result: Pass.
- IT-13 — POST /request/review/accepted/:id: Accept a valid received request. Expected: 200 OK. Result: Pass.
- IT-14 — POST /request/review/accepted/:id: Accept request belonging to another user. Expected: 401 Unauthorized. Result: Pass.
- IT-15 — GET /user/feed: Fetch feed for authenticated user. Expected: 200 OK + array. Result: Pass.
- IT-16 — GET /user/connections: Fetch connections after accepting request. Expected: 200 OK + array. Result: Pass.
- IT-17 — GET /chat/:targetUserId: Fetch chat between connected users. Expected: 200 OK + chat doc. Result: Pass.

SCREENSHOTS

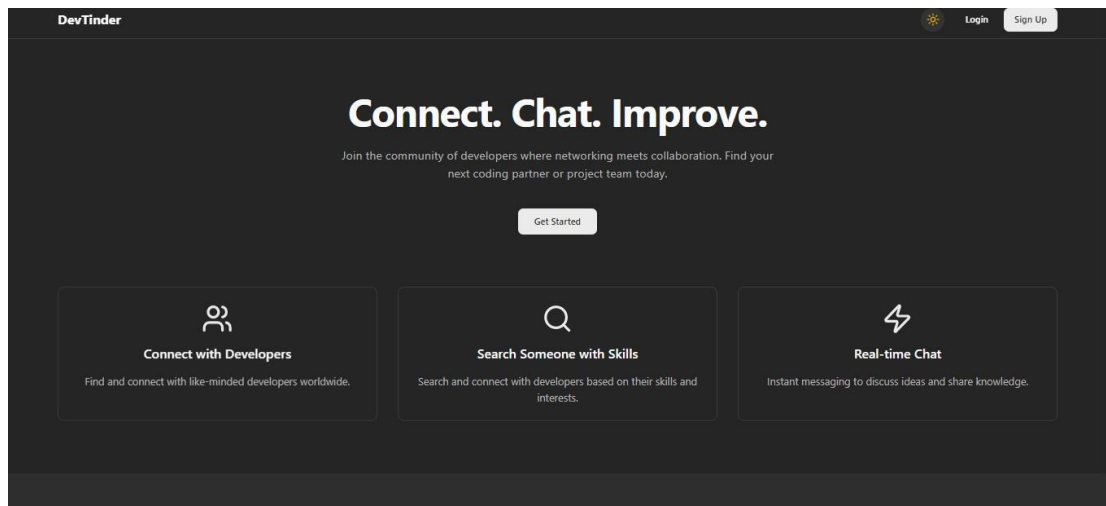


Figure 1: Home Page of DevTinder

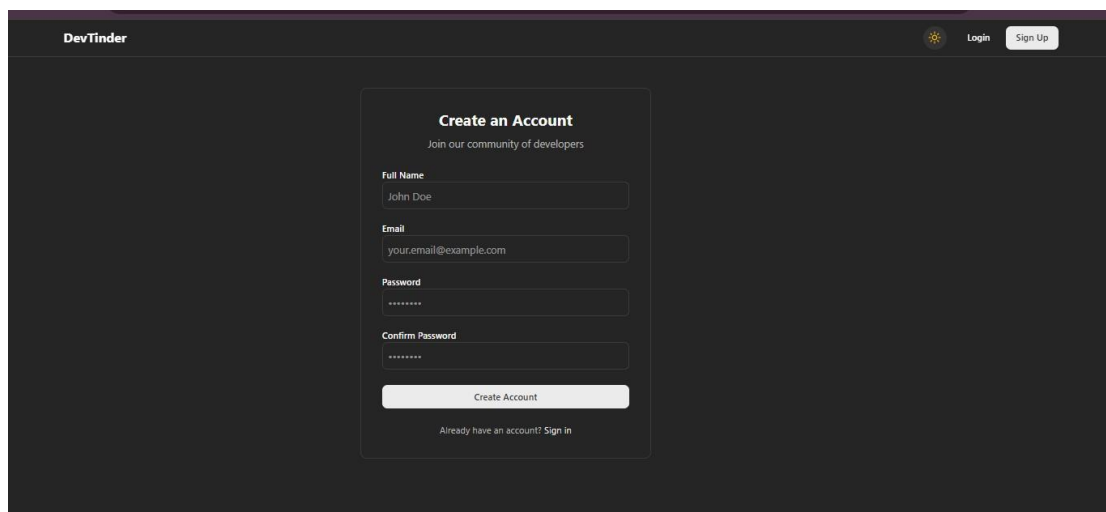


Figure 2: Create Account Page

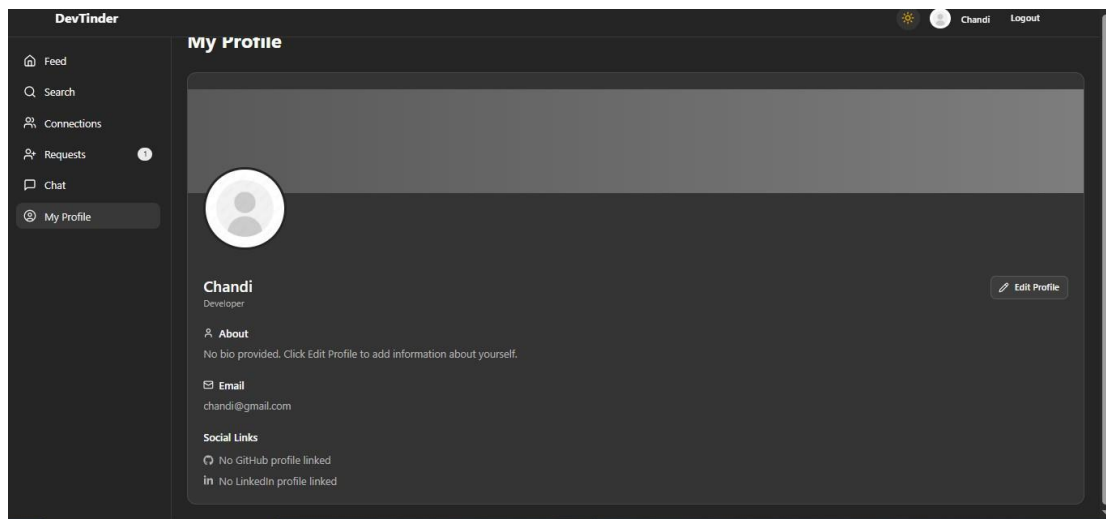


Figure 3: My Profile Page

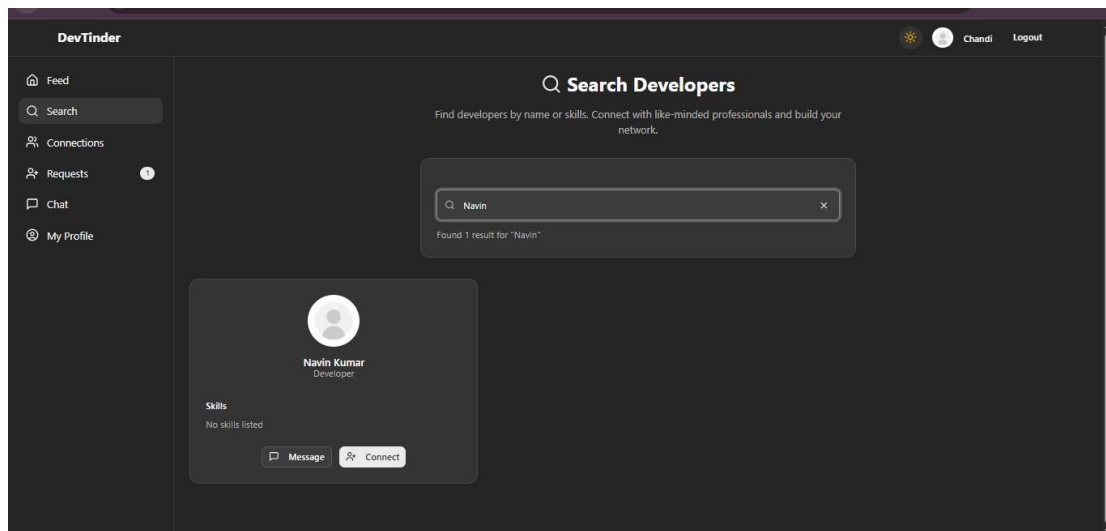


Figure 4: Search to Find Developers

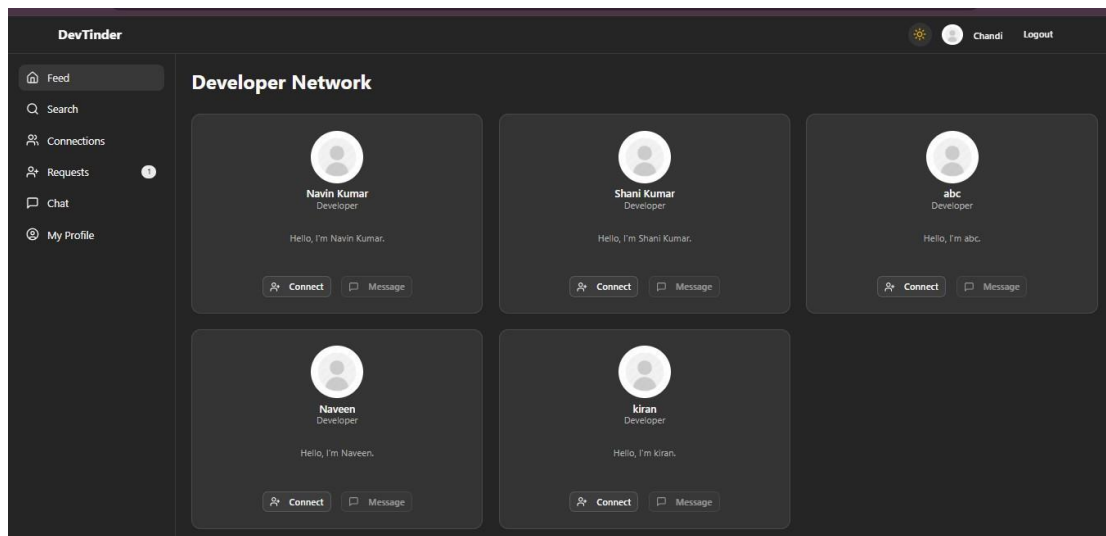


Figure 5: List of Developers

CHAPTER 12: PROJECT FOLDER STRUCTURE

12.1 Backend Folder Structure

The backend follows a domain-driven modular structure. The src/ directory contains all application source code organised into config, models, routes, middlewares, and utils subdirectories. Each subdirectory has a single, clearly defined responsibility.

```
index.js:-
require("dotenv").config()

const express = require('express');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const http = require('http');
const initializeSocket = require("./utils/socket");
const app = express();

const server = http.createServer(app);

// Connect to database
require('./config/database');

// Middleware
app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true,
}));

app.use(express.json());
app.use(cookieParser());

// Routes
app.use('/api/users', require('./routes/userRoutes'));
app.use('/api/connections', require('./routes/connectionRoutes'));
app.use('/api/messages', require('./routes/messageRoutes'));
```



```
initializeSocket(server);
```

```
// Start server
```

```
server.listen(process.env.PORT, () => {  
  console.log(`Server is running on port ${process.env.PORT}`);  
});
```

socket.js :-

```
const socket = require('socket.io');  
const crypto = require('crypto');  
const Chat = require('../models/Chat');  
const Connection = require('../models/Connection');  
  
const getSecretRoomId = (userId, targetUserId) => {  
  return crypto.createHash("sha256").update([userId, targetUserId].sort().join("$")).digest("hex");  
};  
  
const initializeSocket = (server) => {  
  const io = socket(server, {  
    cors: {  
      origin: process.env.FRONTEND_URL,  
      credentials: true,  
    },  
  });  
  
  io.on("connection", (socket) => {  
    console.log(`User connected: ${socket.id}`);  
  
    // Join a room (user's ID) to receive messages  
    socket.on('join_room', (roomId) => {  
      socket.join(roomId);  
      console.log(`User ${socket.id} joined room: ${roomId}`);  
    });  
  
    // Join a chat room with another user  
    socket.on('join_chat', ({ userId, targetUserId }) => {  
      const roomId = getSecretRoomId(userId, targetUserId);  
      socket.join(roomId);  
      console.log(`User ${socket.id} joined chat room: ${roomId}`);  
    });  
  });  
};
```

```

// Handle sending messages
socket.on('send_message', async (data) => {
  try {
    const { sender, receiver, content, timestamp, _id } = data;
    console.log(`Message sent from ${sender} to ${receiver}:`, content);

    // Validate required fields
    if (!sender || !receiver || !content) {
      console.log('Invalid message data:', data);
      return;
    }

    // Check if users are connected
    const connection = await Connection.findOne({
      $or: [
        { requester: sender, recipient: receiver, status: 'accepted' },
        { requester: receiver, recipient: sender, status: 'accepted' }
      ]
    });

    if (!connection) {
      console.log('Users are not connected, message rejected');
      socket.emit('message_error', { error: 'Users are not connected' });
      return;
    }

    // Create a new message object
    const newMessage = {
      senderId: sender,
      text: content.trim(),
      timestamp: timestamp ? new Date(timestamp) : new Date()
    };

    // Find or create a chat between these users
    const participants = [sender, receiver];

    // Try to find existing chat
    let chat = await Chat.findOne({
      participants: { $all: participants }
    });

    // If no chat exists, create a new one

```

```

if (!chat) {
  chat = new Chat({
    participants,
    messages: [newMessage]
  });
} else {
  // Add message to existing chat
  chat.messages.push(newMessage);
}

// Save the chat
await chat.save();

// Get the newly added message (last one in the array)
const savedMessage = chat.messages[chat.messages.length - 1];

// Create the message payload for the receiver
const messagePayload = {
  _id: savedMessage._id,
  senderId: savedMessage.senderId,
  text: savedMessage.text,
  timestamp: savedMessage.timestamp,
  sender: savedMessage.senderId // For compatibility
};

console.log(`Emitting message to receiver: ${receiver}`);

// Only emit to the recipient's room, not back to the sender
// This prevents duplicate messages
io.to(receiver).emit('receive_message', messagePayload);

// Confirm message sent to sender
socket.emit('message_sent', {
  _id: savedMessage._id,
  tempId: _id, // Original temp ID for frontend matching
  timestamp: savedMessage.timestamp
});

```

12.2 Frontend Folder Structure

The frontend is organised with `src/Components/` for React components and `src/utlis/` for Redux slices, the store, and shared constants. This makes it intuitive to find and modify any part of the application.

`main.jsx:-`

```
import { StrictMode } from "react";

import { createRoot } from "react-dom/client";

import "./index.css";

import App from "./App.jsx";

import { Provider } from "react-redux";

import { store } from "./store/store";

createRoot(document.getElementById("root")).render(

  <StrictMode>

    <Provider store={store}>

      <App />

    </Provider>

  </StrictMode>

);
```

`App.jsx:-`

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { useDispatch } from 'react-redux';
import { useEffect } from 'react';

// Components
import ErrorBoundary from './components/ErrorBoundary';
import ProtectedRoute from './components/ProtectedRoute';

// Layout
```

```

import MainLayout from './components/layout/MainLayout';

// Redux actions
import { getCurrentUser } from './store/slices/authSlice';

// Pages
import LandingPage from './pages/LandingPage';
import LoginPage from './pages/LoginPage';
import SignupPage from './pages/SignupPage';
import TermsPage from './pages/TermsPage';
import PrivacyPage from './pages/PrivacyPage';
import NotFoundPage from './pages/NotFoundPage';

// Dashboard pages
import DashboardPage from './pages/DashboardPage';
import ConnectionsPage from './pages/ConnectionsPage';
import RequestsPage from './pages/RequestsPage';
import ChatPage from './pages/ChatPage';
import UserProfilePage from './pages/UserProfilePage';
import ProfilePage from './pages/ProfilePage';
import SearchPage from './pages/SearchPage';

// Initialize theme from localStorage or system preference
const initializeTheme = () => {
  const savedTheme = localStorage.getItem('theme');

  if (savedTheme === 'dark' ||
    (!savedTheme && window.matchMedia('(prefers-color-scheme: dark)').matches)) {
    document.documentElement.classList.add('dark');
  } else {
    document.documentElement.classList.remove('dark');
  }
};

```

CHAPTER 13: CONCLUSION AND FUTURE SCOPE

13.1 Conclusion

DevTinder is a fully functional, well-architected, and feature-rich full-stack web application that successfully demonstrates modern web development practices in a practical context. The project has met all eight objectives stated in Chapter 1: delivering a complete developer networking platform with profile management, feed-based discovery, a bidirectional connection lifecycle, real-time one-on-one chat, and a robust security architecture - with no payment integration anywhere in the system.

The frontend, built with React 19 and Redux Toolkit, provides a responsive, accessible, and performant user experience across both desktop and mobile browsers. The component-based architecture, centralised Redux state management, and Tailwind CSS ensure the application is visually consistent, maintainable, and readily extensible. React Router v7 delivers seamless client-side navigation.

The backend, implemented with Node.js and Express.js, provides a clean, modular RESTful API with six independent route modules, comprehensive middleware, and well-encapsulated business logic. Mongoose schema validation enforces data integrity at the database layer, while the custom auth middleware provides consistent authentication across all protected endpoints.

The real-time chat system, powered by Socket.io with a deterministic room ID design, provides reliable sub-second message delivery with full persistence in MongoDB. The security architecture - combining HTTP-only JWT cookies, bcrypt password hashing, whitelist-based input validation, CORS restrictions, and environment variable management - adheres to industry best practices.

The testing phase, covering 38 test cases across unit, integration, and end-to-end levels, produced a 100% pass rate, validating the correctness and robustness of all features. This project was successfully developed, tested, and demonstrated at Cambridge Institute of Technology, Ranchi, under the guidance of Prof. Deepak Kumar Verma, as part of the 8th Semester Final Year Project for the Bachelor of Engineering in Computer Science and Engineering.

13.2 Future Scope

13.2.1 Cloud Deployment

The application can be deployed to cloud infrastructure - the Node.js backend on AWS EC2, the React frontend on S3 with CloudFront CDN, and MongoDB on MongoDB Atlas. A GitHub Actions CI/CD pipeline would automate testing and deployment on every push.

13.2.2 Push Notifications

Integration of browser Push Notifications using the Web Push API and a service worker would alert users to new connection requests, accepted connections, and incoming chat messages, improving engagement without requiring the user to have the app open.

13.2.3 Advanced Developer Search and Filtering

A dedicated search interface with full-text search on skills and bio (using MongoDB Atlas Search), combined with filter controls for technology stack and experience level, would allow developers to find specific collaborators efficiently.

13.2.4 Group Chat and Community Rooms

Extending the real-time messaging system to support multi-user group chat rooms organised around programming languages or frameworks (e.g., a 'React.js Developers' room) would enable community-scale interactions beyond one-on-one conversations.

13.2.5 Video and Voice Calling

Integration of WebRTC-based peer-to-peer video and audio calling between connected developers would create a comprehensive synchronous collaboration platform for pair programming, technical interviews, and mentoring sessions.

13.2.6 React Native Mobile Application

A React Native mobile application sharing the same Node.js/Express.js backend API would extend DevTinder to iOS and Android users with native push notifications and camera integration for profile photos.

13.2.7 OAuth 2.0 Social Sign-In

Adding GitHub OAuth, Google OAuth, and LinkedIn OAuth sign-in would reduce registration friction. GitHub OAuth could additionally import the user's public repositories, primary programming languages, and pinned projects into their DevTinder profile.

13.2.8 AI-Powered Recommendations

A machine learning recommendation engine that analyses skills, connection patterns, and interaction history could produce a personalised, ranked developer feed using collaborative filtering techniques, significantly improving connection quality and user satisfaction.

--- End of Report ---

Navin Kumar (22050440067) | Chandni Charan Mahto (22050440034)
8th Semester, B.Tech, CSE | Cambridge Institute of Technology, Ranchi | 2025-26

REFERENCES

- [1] React Official Documentation. The React Team. Available: <https://react.dev>. Accessed: 2024.
- [2] Node.js Official Documentation. OpenJS Foundation. Available: <https://nodejs.org/en/docs>. Accessed: 2024.
- [3] Express.js Official Documentation. Available: <https://expressjs.com>. Accessed: 2024.
- [4] MongoDB Official Documentation. MongoDB Inc. Available: <https://www.mongodb.com/docs>. Accessed: 2024.
- [5] Mongoose ODM Documentation. Available: <https://mongoosejs.com/docs>. Accessed: 2024.
- [6] Socket.io Official Documentation. Available: <https://socket.io/docs/v4>. Accessed: 2024.
- [7] Redux Toolkit Official Documentation. Available: <https://redux-toolkit.js.org>. Accessed: 2024.
- [8] React Router v7 Documentation. Available: <https://reactrouter.com>. Accessed: 2024.
- [9] Tailwind CSS Documentation. Available: <https://tailwindcss.com/docs>. Accessed: 2024.
- [10] DaisyUI Component Library Documentation. Available: <https://daisyui.com>. Accessed: 2024.
- [11] JSON Web Tokens Introduction. Available: <https://jwt.io/introduction>. Accessed: 2024.
- [12] AWS Simple Email Service Developer Guide. Amazon Web Services. Available: <https://docs.aws.amazon.com/ses>. Accessed: 2024.
- [13] Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media, Sebastopol, CA.

- [14] Brown, E. (2019). Web Development with Node and Express (2nd ed.). O'Reilly Media.
- [15] Chodorow, K. (2013). MongoDB: The Definitive Guide (2nd ed.). O'Reilly Media, Sebastopol, CA.
- [16] Vite Build Tool Documentation. Available: <https://vitejs.dev>. Accessed: 2024.
- [17] bcrypt npm Package Documentation. Available: <https://www.npmjs.com/package/bcrypt>. Accessed: 2024.
- [18] validator.js Documentation. Available: <https://github.com/validatorjs/validator.js>. Accessed: 2024.
- [19] node-cron Documentation. Available: <https://github.com/node-cron/node-cron>. Accessed: 2024.
- [20] Vite Documentation. Available: <https://vitejs.dev/guide/>. Accessed: 2024.